

Guía de acceso a la base de datos para el equipo

Para: Herlin Echeverry, Pablo Arango y Bryan Martínez. **Administra:** William Ortiz (dueño del repositorio y del proyecto de Supabase).

Fecha: 4 de octubre de 2026.

Regla de oro: nunca compartas por chat, correo ni en el repositorio una contraseña, un token o la clave secreta (`service_role`). El repositorio es privado, pero igual no se sube ningún secreto.

0. Quién hace qué

Paso	William (administrador)	Cada integrante
1. GitHub	Ya envió la invitación	Aceptarla, clonar el repo y cerrar su copia pública
2. Pruebas locales	—	<code>npm install</code> y <code>npm test</code> en <code>db/</code>
3. Panel de Supabase	Invitar por correo con el rol Developer	Aceptar e iniciar sesión con GitHub
4. Trabajo en el frontend	Entregar la URL y la clave pública	Crear <code>maqueta3d/.env.local</code>
5. Cambios a la base	Aplicarlos con <code>db push</code> cuando se fusiona el PR	Escribir la migración, probarla y abrir un PR

1. GitHub: entrar al repositorio privado

- Acepta la invitación.** Llega a tu correo, o la ves en <https://github.com/leonidas452528/hackaton-camaracomercio2026/invitations>. Vence a los 7 días; si se vence, pídele a William otra.
 - Bryan: ya aceptó.
 - Herlin y Pablo: pendiente.
- Antes de cerrar tu copia, revisa si tiene cambios que no estén en el repo de William.** Cuando el repo pasó a privado, GitHub dejó **públicas** las copias que hicieron con fork: la de Pablo quedó como repo independiente, y las de Herlin y Bryan como forks de la de Pablo. Si tienes trabajo propio, súbelo como rama al repo de William (paso 3) o pásaselo.
- Clona el repo privado** y trabaja siempre desde ahí:

```
git clone https://github.com/leonidas452528/hackaton-camaracomercio2026.git
cd hackaton-camaracomercio2026
```

Si ya tenías un clon de tu fork, puedes reutilizarlo cambiando el remoto:

```
git remote set-url origin https://github.com/leonidas452528/hackaton-camaracomercio2026.git
git fetch origin
```

4. **Cierra tu copia pública:** en tu repo hackaton-camaracomercio2026, *Settings* → *General* → *Danger Zone* → *Change visibility* → *Private* (o *Delete this repository*). Mientras siga pública, cualquiera puede ver la idea tal como estaba el 25 o 26 de septiembre.

2. Correr la base de datos en tu computador (sin Supabase)

Necesitas Node.js 20 o superior. No necesitas Docker ni una cuenta de Supabase.

```
cd hackathon-cali-2026/db
npm install
npm test
```

Debes ver "Todas las pruebas pasan" dos veces: 69 pruebas del SQL y 17 de la carga. Corre PostgreSQL con PostGIS dentro de Node (PGlite) y carga los datos reales en memoria. Con esto puedes probar cualquier cambio sin tocar el proyecto compartido.

3. Panel de Supabase: ver el proyecto *dev*

1. **William** te invita: *Organization settings* → *Team* → *Invite member*, con tu correo y el rol **Developer**.
2. **Tú** aceptas la invitación del correo e inicias sesión en <https://supabase.com/dashboard> con **Continue with GitHub**, con la misma cuenta del repo.
3. Abres el proyecto: <https://supabase.com/dashboard/project/rqxltixtsiqsaxapdue>

Cuidado: el editor de tablas y el editor SQL del panel trabajan con privilegios de administrador y **se saltan las reglas de seguridad por fila**. Úsalos para mirar, no para cambiar datos ni el esquema: todo cambio va por migración (sección 5). En dev no hay ni puede haber datos personales reales.

4. Conectar el frontend (*maqueta3d*) a la base

1. William te pasa dos valores **públicos**, que están en *Project Settings* → *API Keys*:
 - La URL: <https://rqxltixtsiqsaxapdue.supabase.co>
 - La clave **publishable** (o anon). Nunca la secret ni la service_role: esa da control total y jamás va en el navegador.
2. Crea el archivo `maqueta3d/.env.local`. Ya está en `.gitignore`, así que no se sube:

```
VITE_SUPABASE_URL=https://rqxltixtsiqsaxapdue.supabase.co
VITE_SUPABASE_ANON_KEY=la-clave-publishable
```

3. En el código, consulta **solo** el esquema api. El contrato está en `db/docs/CONTRATO_API.md`:

```
supabase.schema('api').from('espacios').select('*')
supabase.schema('api').rpc('registrar_medicion', { ... })
```

4. Para listas y mapas usa `api.espacios`; la ficha completa (`api.espacio_ficha`) pídela por id.

Cuentas de demostración (ya creadas): hay 13, una por rol (lista en `db/docs/ROLES_Y_PERMISOS.md` §3),

con correos como sgred.demo@example.org. Las contraseñas las tiene **solo William** y se piden por un canal seguro, nunca por el grupo. Recuerda:

- Son cuentas **compartidas y ficticias**: no cambies su contraseña ni sus datos.
- sgred, coordinacion y plataforma necesitan inscribir un factor TOTP (por ejemplo, Google Authenticator) para decidir activaciones o gestionar usuarios; sin eso, la base lo bloquea. La pantalla de inscripción todavía no existe en la app.

5. Proponer un cambio a la base de datos

Nadie cambia la base desde el panel. Un cambio sigue estos pasos:

1. Crea una rama:

```
git switch main && git pull
git switch -c db/mi-cambio
```

2. Crea una migración nueva. Nunca edites una migración ya aplicada:

```
cd db
npx supabase migration new describe_el_cambio
```

Esto crea db/supabase/migrations/<fecha>_describe_el_cambio.sql. Escribe ahí el SQL.

3. **Agrega o ajusta una prueba** en db/tests/ y corre npm test hasta que todo pase.
4. **Sube la rama y abre un pull request** hacia main:

```
git push -u origin db/mi-cambio
gh pr create --base main
```

5. **El responsable de la base de datos** revisa, fusiona y aplica con npx supabase db push. Solo esa persona necesita la contraseña de la base; si cambia de responsable, William la restablece en el panel y se la entrega por un canal seguro.

6. Si te toca administrar la base (solo el responsable)

1. Instala las dependencias: cd db && npm install. Trae la CLI de Supabase, versión fija.
2. En una **terminal normal** (no dentro de un asistente, porque necesita una terminal interactiva):

```
cd hackathon-cali-2026/db
npx supabase login
npx supabase link --project-ref rqxltixtsiqsaxpdue
```

login abre el navegador; link pide la contraseña de la base, que se escribe solo ahí.

3. Para conectarte por fuera de la CLI (por ejemplo, la carga de datos), usa siempre la verificación TLS con la CA pública del repo:

```
read -rs PGPASSWORD && export PGPASSWORD
```

```
DATABASE_URL="postgresql://postgres.rqxltixtsiqsakxapdue@aws-0-us-east-1.pooler.supabase.com:5432/postgres?sslmode=verify-full&sslrootcert=$PWD/certs/supabase-prod-ca-2021.crt" npm run cargar  
unset PGPASSWORD
```

4. Para aplicar migraciones, mira primero qué entraría y después aplica:

```
npx supabase db push --dry-run  
npx supabase db push  
npx supabase migration list
```

7. Reglas del equipo

- **Sin datos personales.** Ni nombres, ni cédulas, ni teléfonos, ni datos de menores. Las cuentas de prueba son ficticias y usan correos @example.org.
- **Sin secretos en el repo, el chat ni capturas de pantalla.**
- **Lo simulado se marca como simulado.** Lo que se crea con cuentas de demostración queda con `es_simulado`.
- **Cada norma o cifra nueva va con su fuente** en `docs/cumplimiento/FUENTES.md`.
- **Toda duda de permisos se resuelve en la base, no en el frontend.** Si algo "no deja", probablemente la base lo está protegiendo a propósito: revisa `db/docs/ROLES_Y_PERMISOS.md` antes de cambiarlo.